Rockchip Linux USB Host UVC Troubleshooting

ID: RK-PC-YF-493

Release Version: V1.1.0

Release Date: 2026-06-22

Security Level: □Top-Secret □Secret □Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD. (“ROCKCHIP”) DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS, MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2026. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, Software Boulevard Fuzhou, Fujian, PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document provides troubleshooting methods and solutions for common USB Host UVC (USB Video Class) issues based on the Linux kernel. The purpose is to assist software development engineers and technical support engineers in quickly locating and resolving UVC-related issues.

Chip Models	Kernel Version
All Rockchip Chips (excluding MCU)	Linux-4.19 and above

Intended Audience

This document (this guide) is mainly intended for:

- Technical Support Engineers
- Software Development Engineers

Revision History

Date	Version	Author	Modification Description
2023-11-27	V1.0.0	Wu Liangfeng, Long Yue	Initial version
2026-06-22	V1.1.0	Wang Mingcheng	Formatting fixes

Contents

Rockchip Linux USB Host UVC Troubleshooting

- 1. UVC Communication Process
- 2. Common Debugging Tools for UVC
  - 2.1 USB Protocol Analyzer
  - 2.2 USB Monitoring and Analysis Tools
  - 2.3 v4l2-ctl Tool
- 3. UVC Kernel Debugging Interface
  - 3.1 UVC Driver Debug Interface
  - 3.2 V4L2 Debugging Interface
  - 3.3 Debugging Interface for Dynamic Initialization of USB Controllers
- 4. UVC Debugging Methods
  - 4.1 Method for Binding USB Interrupts to a Specific Core
  - 4.2 Method for Disabling UVC Auto Suspend
  - 4.3 Method for Disabling CPU IDLE
  - 4.4 Method for Setting CPU to High Frequency
  - 4.5 Method for Setting DDR to High Frequency
  - 4.6 Method for Parsing UVC Image Frames
  - 4.7 Methods for Optimizing UVC Driver
- 5. UVC Common Troubleshooting Methods
  - 5.1 UVC Preview Failure Issues
  - 5.2 Unstable UVC Preview Frame Rate
  - 5.3 UVC Preview Artifacts
  - 5.4 Insufficient Bandwidth Issues in Multi-UVC Scenarios
  - 5.5 UVC Preview Latency Issues
  - 5.6 UVC EMI Issues
- 6. References

1. UVC Communication Process

As shown in Figure 1, it is the UVC communication process based on the Linux kernel. The left side of Figure 1 describes the UVC-related modules on the USB Device side and the corresponding kernel code paths. The right side of Figure 1 describes the UVC-related modules on the USB Host side and the corresponding kernel code paths. This document mainly explains the troubleshooting methods for UVC drivers on the USB Host side, and the debugging methods provided are mainly based on the Rockchip platform Linux-5.10 kernel; therefore, they may not be applicable to all kernel versions, nor are they suitable for troubleshooting UVC application-related issues. From the process on the right side of Figure 1, it can be seen that applications on the UVC Host side can communicate data with the kernel UVC device in two ways: one is through the V4L2 framework calling the UVC driver's interface for communication; the other is through libusb calling the USB generic driver Devio's interface for communication. Both communication methods are supported on the Rockchip platform, and the actual communication method used is determined by the UVC application.

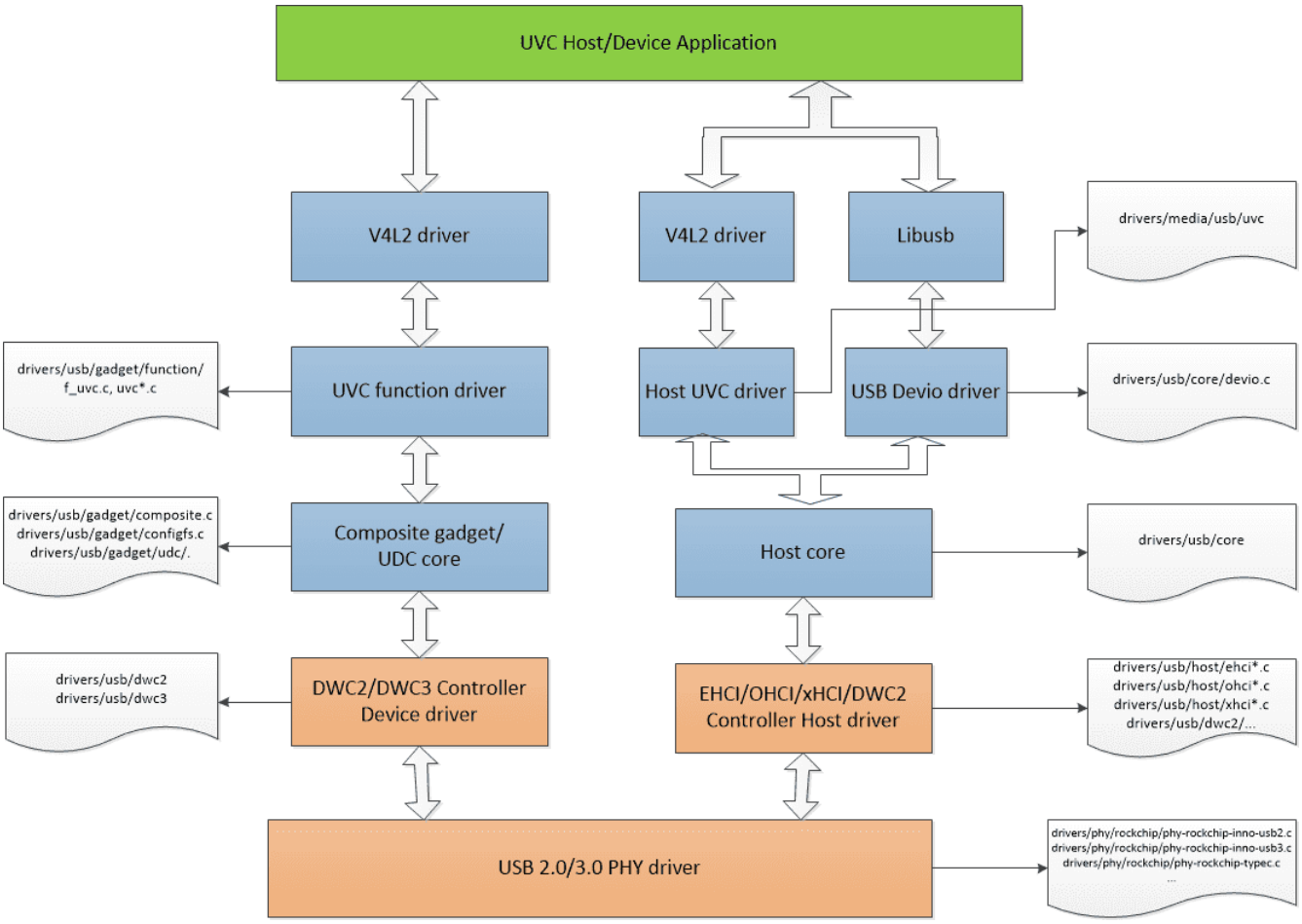


Figure 1 Linux UVC Communication Process

2. Common Debugging Tools for UVC

2.1 USB Protocol Analyzer

USB Protocol Analyzers can be used for real-time capture of communication data on the USB bus, enabling USB data packet capture and protocol analysis. As shown in Figure 2 below, this is the common connection method for USB 2.0 protocol analyzers. When using them, the USB analyzer should be daisy-chained between the USB Device and Host, and corresponding protocol analysis software must be installed on the computer. USB protocol analyzers are the most effective and direct method for pinpointing whether the issue lies on the Device side or the Host side.

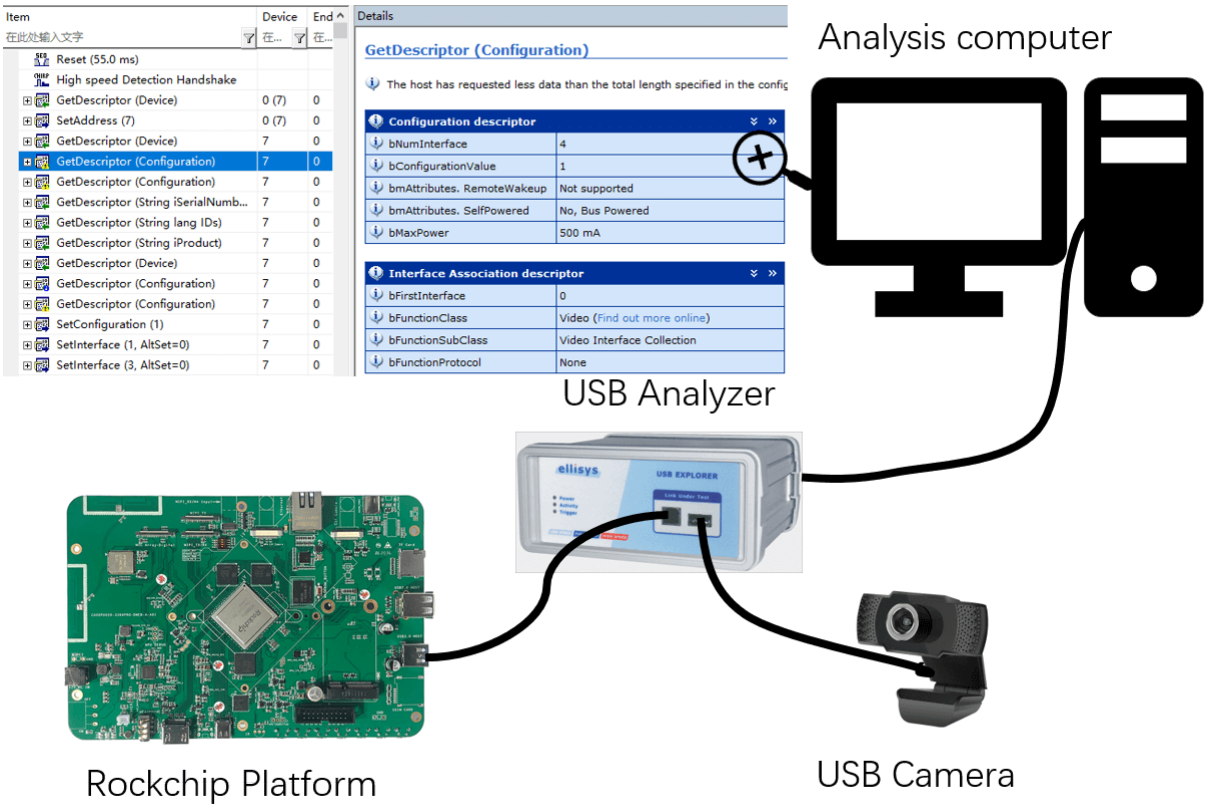


Figure 2 Connection diagram of USB Protocol Analyzer

- USB Protocol Analyzers can be used to analyze several common issues with UVC:

1. Failure to open UVC preview image;
2. Delay issues with UVC preview image;
3. UVC preview image freezing issues;
4. UVC frame rate statistics and image data parsing;
5. Issues with UVC driver loading failure;

- USB Protocol Analyzers are not suitable for analyzing several common issues with UVC:

1. USB signal quality issues;
2. Abnormal disconnection issues with UVC;
3. EMI errors triggered during UVC communication with USB controller;

High-bandwidth oscilloscopes can be used to measure USB eye diagram signal quality, USB high-speed handshake signals, and USB VBUS power collapse conditions, to analyze USB signal issues, abnormal disconnection issues, and EMI issues.

2.2 USB Monitoring and Analysis Tools

USB monitoring, or usbmon, is a USB packet capture tool provided by the Linux kernel, used to capture data exchanged between the USB core layer and device class drivers (such as UVC drivers), as well as data exchanged between the USB core layer and USB Host Controller drivers. By capturing packets with usbmon and using packet analysis tools (such as vusb-analyzer or Wireshark) to parse the USB communication protocol, one can visually observe the processing flow of kernel UVC data, aiding in the analysis and troubleshooting of UVC driver issues and USB controller driver issues.

The usbmon driver module is located in the kernel directory `drivers/usb/mon/`. To enable the usbmon feature:

1. Enable CONFIG_USB_MON in the kernel.
2. Mount the debugfs filesystem.

   ```
   mount -t debugfs none_debugs /sys/kernel/debug
   ```

   If it prompts busy, it indicates that the debugfs filesystem is already mounted by default on the system.

   In most Rockchip SDK distribution versions, usbmon is supported by default. To check if the system supports usbmon:

   ```
   ls /sys/kernel/debug/usb/usbmon/
   ```

Data packet analysis tool: Download address for vusb-analyzer is https://vusb-analyzer.sourceforge.net/download.html.

Ubuntu installation method: `sudo apt-get install vusb-analyzer`.

Taking the application scenario of connecting a UVC peripheral to RK3588 EVB1 USB2 HOST0 as an example:

1. Check the USB bus numbers recognizable by usbmon.

   The numbers 1/2/3, etc., represent the USB buses available on the current platform, with each USB device hanging under a bus; the number 0 indicates capturing packets from all buses; the letters s/t/u following the numbers represent the data format for packet saving, with u format commonly used.

   ```
   console:/sys/kernel/debug/usb/usbmon # ls
   0s  1s  1u  2t  3s  3u  4t  5s  5u  6t
   0u  1t  2s  2u  3t  4s  4u  5t  6s  6u
   ```

2. Identify the bus number used by the device to be monitored.

   Execute the lsusb command to distinguish based on the vendor ID and product ID.

```
console:/sys/kernel/debug/usb/usbmon # lsusb
Bus 005 Device 001: ID 1d6b:0002
Bus 003 Device 001: ID 1d6b:0001
Bus 002 Device 005: ID 046d:0823
Bus 001 Device 001: ID 1d6b:0002
Bus 006 Device 001: ID 1d6b:0003
Bus 004 Device 001: ID 1d6b:0001
Bus 002 Device 001: ID 1d6b:0002
```

Here, Bus 002 indicates bus number 2. The ID 046d:0823 corresponds to the vendor ID and product ID of the USB device to be monitored, indicating that the monitored USB device uses bus number 2.

3. Use usbmon to capture communication packets.

```
cat  /sys/kernel/debug/usb/usbmon/2u > /data/usbmon.mon
```

4. Analyze the packets with the vusb-analyzer tool.

```
vusb-analyzer usbmon.mon
```

2.3 v4l2-ctl Tool

The v4l2-ctl tool is used to control video devices based on the Linux V4L2 framework (/dev/video*). Through the v4l2-ctl tool, various operations can be performed on video devices, including: reading the formats supported by the device, capturing images at specific resolutions and formats, etc. The Linux UVC driver is implemented based on the V4L2 framework. When debugging UVC issues, if there is suspicion related to UVC applications, the v4l2-ctl tool can be used to operate UVC devices to exclude the impact of UVC applications.

1. **v4l2-ctl --list-devices**

   List all devices (including all video devices registered through the V4L2 framework)

   Example:

```
console:/ # v4l2-ctl --list-devices
UVC Camera (046d:0823) (usb-fc800000.usb-1):
        /dev/video21
        /dev/video22
```

   A USB camera corresponds to two devices: one for image/video capture and one for metadata capture.

2. **v4l2-ctl --list-formats-ext --device path/to/video_device**

   List the preview-supported formats for a specified device

   Example:

```
console:/ # v4l2-ctl --list-formats-ext --device /dev/video21
ioctl: VIDIOC_ENUM_FMT
        Index       : 0
```

```
        Type        : Video Capture
        Pixel Format: 'YUYV'
        Name        : YUYV 4:2:2
                Size: Discrete 640x480
                        Interval: Discrete 0.033s (30.000 fps)
                        Interval: Discrete 0.042s (24.000 fps)
                        Interval: Discrete 0.050s (20.000 fps)
                        Interval: Discrete 0.067s (15.000 fps)
                        Interval: Discrete 0.100s (10.000 fps)
                        Interval: Discrete 0.133s (7.500 fps)
                        Interval: Discrete 0.200s (5.000 fps)
         ......
```

3. **v4l2-ctl --all --device path/to/video_device**

   Obtain all information for a specified device

   Example:

```
console:/ # v4l2-ctl --all --device /dev/video21
Driver Info:
        Driver name      : uvcvideo
        Card type        : UVC Camera (046d:0823)
        Bus info         : usb-fc800000.usb-1
        Driver version   : 5.10.198
        Capabilities     : 0x84a00001
                Video Capture
                Metadata Capture
                Streaming
                Extended Pix Format
                Device Capabilities
        Device Caps      : 0x04200001
                Video Capture
                Streaming
                Extended Pix Format
        ......
```

4. **v4l2-ctl --device path/to/video_device --set-fmt-video=width=width,height=height,pixelformat=MJPG --stream-mmap --stream-to=path/to/output.jpg --stream-count=1**

   Capture images from a specific device at a specific resolution and format

   Example:

   Capture 1 frame of image from the video21 device at 1920x1080 resolution in MJPG format and save it in the /sdcard/DCIM/ path.

```
v4l2-ctl --device /dev/video21 --set-fmt-video=width=1920,height=1080,pixelformat=MJPG --stream-mmap --stream-to=/sdcard/DCIM/output_1920.jpg --stream-count=1
```

5. **v4l2-ctl --device path/to/video_device --set-fmt-video=width=width,height=height,pixelformat=format --stream-mmap --stream-to=path/to/output --stream-count=number_of_frames_to_capture**

   Capture streaming from a specific device at a specific resolution

Example 1:

Capture 100 frames of image stream from the video21 device at 1920x1080 resolution in MJPG format and save it in the /sdcard/DCIM/ path.

```
v4l2-ctl --device /dev/video21 --set-fmt-video=width=1920,height=1080,pixelformat=MJPG --stream-mmap --stream-to=/sdcard/DCIM/output.mjpg --stream-count=100
```

Example 2:

Continuously capture image stream from the video21 device at 1920x1080 resolution in MJPG format.

```
v4l2-ctl --device /dev/video21 --set-fmt-video=width=1920,height=1080,pixelformat=MJPG --stream-mmap
```

3. UVC Kernel Debugging Interface

3.1 UVC Driver Debug Interface

The UVC driver module is located in the kernel `drivers/media/usb/uvc` and provides a debugging interface based on system modules, which can be used to dynamically enable trace information for the UVC driver, with trace information functionality defaulting to off.

`/sys/module/uvcvideo/parameters/trace`

To dynamically enable trace:

`echo 0xffff > /sys/module/uvcvideo/parameters/trace`

The information print level is KERN_DEBUG, and the trace information can be viewed by executing the command dmesg. You can also modify the kernel's printk level, which defaults to outputting to the serial console terminal:

`echo 8 > /proc/sys/kernel/printk`

To dynamically disable trace:

`echo 0 > /sys/module/uvcvideo/parameters/trace`

Taking the RK3588 USB2 HOST0 connection to UVC peripheral and opening the preview image as an example, the printed information is as follows:

```
[   60.780597][  T477] uvcvideo: uvc_v4l2_open
[   60.843298][  T477] uvcvideo: Resuming interface 2
[   60.843318][  T477] uvcvideo: Resuming interface 3
[   60.852317][  T477] uvcvideo: uvc_v4l2_release
[   60.855127][  T477] uvcvideo: uvc_v4l2_open
[   60.877415][  T477] uvcvideo: Trying format 0x47504a4d (MJPG): 2592x1944.
[   60.877525][  T477] uvcvideo: Using default frame interval 100000.0 us (10.0 fps).
[   60.910575][  T477] uvcvideo: Device requested 3060 B/frame bandwidth.
[   60.910622][  T477] uvcvideo: Selecting alternate setting 11 (3060 B/frame bandwidth).
[   60.913504][  T477] uvcvideo: Allocated 5 URB buffers of 32x3060 bytes each.
[   60.913836][  T477] uvcvideo: uvc_v4l2_poll
[   62.010310][    C0] uvcvideo: Frame complete (EOF found).
[   62.010438][    C0] uvcvideo: frame 1 stats: 1111/763/1876 packets, 1/1111/1876 pts (early
initial), 1875/1876 scr, last pts/stc/sof 223854994/228661160/514
```

3.2 V4L2 Debugging Interface

The Linux UVC driver is based on the V4L2 framework, implementing operations such as queue buf and dequeue buf with UVC applications.

The V4L2 framework implements tracepoints[1] based on the kernel trace framework, which can be used to track the processes of queue buf, dequeue buf, and buf done.

The kernel needs to enable the following Trace-related configurations:

```
Kernel hacking  -->
        [*] Tracers  -->
                [*]   Trace process context switches and events
                [*]   Enable uprobes-based dynamic events
                [*]   Trace gpio events:
```

V4L2 supports the following trace events:

```
/sys/kernel/debug/tracing/events/vb2# ls
enable  filter  vb2_buf_done  vb2_buf_queue  vb2_dqbuf  vb2_qbuf
/sys/kernel/debug/tracing/events/v4l2# ls
enable  v4l2_dqbuf  vb2_v4l2_buf_done   vb2_v4l2_dqbuf
filter  v4l2_qbuf   vb2_v4l2_buf_queue  vb2_v4l2_qbuf
```

For detailed usage, please refer to `/sys/kernel/debug/tracing/README`.

3.3 Debugging Interface for Dynamic Initialization of USB Controllers

`/sys/bus/platform/drivers/[usb_controller_name]/bind, unbind`

Note: The command '[usb_controller_name]' should be replaced with the name of the USB controller corresponding to the chip.

When UVC data transfer encounters anomalies and cannot be automatically recovered, you can try to manually trigger the re-initialization of the USB controller to determine if it is related to USB controller anomalies.

Taking the RK3588 EVB1 platform as an example:

```
/* USB2 HOST0/1 OHCI Controller Driver */
console:/sys/bus/platform/drivers/ohci-platform # ls
bind  fc840000.usb  fc8c0000.usb  uevent  unbind

/* USB2 HOST0/1 EHCI Controller Driver */
console:/sys/bus/platform/drivers/ehci-platform # ls
bind  fc800000.usb  fc880000.usb  uevent  unbind

/* USB3 OTG1_HOST Controller Driver */
console:/sys/bus/platform/drivers/xhci-hcd # ls
bind  uevent  unbind  xhci-hcd.8.auto

/* USB3 OTG0/1 Controller Driver */
console:/sys/bus/platform/drivers/dwc3 # ls
bind  fc000000.usb  fc400000.usb  uevent  unbind

Execute the following commands to re-initialize the OHCI/EHCI controller corresponding to the
USB2 HOST0 interface
echo fc840000.usb > /sys/bus/platform/drivers/ohci-platform/unbind
echo fc800000.usb > /sys/bus/platform/drivers/ehci-platform/unbind
echo fc800000.usb > /sys/bus/platform/drivers/ehci-platform/bind
echo fc840000.usb > /sys/bus/platform/drivers/ohci-platform/bind
```

4. UVC Debugging Methods

4.1 Method for Binding USB Interrupts to a Specific Core

Purpose: This method can be utilized to analyze and resolve issues such as UVC preview screen tearing and unstable UVC frame rates, caused by slow USB controller interrupt processing under high-load scenarios in the system.

The Linux kernel provides the following node for setting the affinity of a CPU and a specific interrupt. The file contains a list of CPUs (in decimal). Note that the CPU core count is represented by numbers starting from 0, such as CPU0, CPU1, etc.

`/proc/irq/[irq_num]/smp_affinity_list`

Taking the RK3588 EVB1 as an example, the method to bind the USB OTG0 controller's interrupt to CPU1 is as follows:

```
console:/ # cat /proc/interrupts | grep dwc
153:         77          0          0          0          0          0          0          0
GICv3 252 Level     dwc3
console:/ # echo 1 > /proc/irq/153/smp_affinity_list
```

4.2 Method for Disabling UVC Auto Suspend

Purpose: This can be used to analyze and resolve compatibility issues with auto suspend for some UVC peripherals on the Rockchip platform.

When debugging such issues, you can dynamically disable the USB autosuspend feature to preliminarily locate the problem using the following command.

```
for i in $(find /sys -name control | grep usb);
do echo on > $i;
echo "echo on > $i";
done;
```

If disabling the auto suspend feature resolves issues encountered during the use of specific UVC peripherals, it indicates that there is an auto suspend compatibility issue with the UVC peripheral on the Rockchip platform. You can modify the USB driver to add the UVC peripheral to the auto suspend blacklist. For example: Adding the UVC peripheral (VID,PID=0x05a3,0x9230) to the blacklist, the Linux-5.10 kernel reference modification is as follows:

```
diff --git a/drivers/usb/core/quirks.c b/drivers/usb/core/quirks.c
index 76ac5d6555ae..d799e93b9a0d 100644
--- a/drivers/usb/core/quirks.c
+++ b/drivers/usb/core/quirks.c
@@ -322,6 +322,9 @@ static const struct usb_device_id usb_quirk_list[] = {
        /* Alcor Micro Corp. Hub */
        { USB_DEVICE(0x058f, 0x9254), .driver_info = USB_QUIRK_RESET_RESUME },

+       /* HD Camera Manufacturer */
+       { USB_DEVICE(0x05a3, 0x9230), .driver_info = USB_QUIRK_AUTO_SUSPEND },
```

The following Table 1 lists information on known UVC peripherals with auto suspend compatibility issues.

Table 1: USB Camera Auto Suspend Blacklist

Vendor ID (VID)	Product ID (PID)	Manufacturer
0x05a3	0x9230	Microdia
0x05a3	0x9320	Microdia
0x0c45	0x6321	Sonix
0x0c45	0x636a	Sonix
0x0c45	0x636b	Sonix
0x0c45	0x64ab	Sonix
0x0c45	0x64ac	Sonix
0x0e8d	0x7668	MediaTek
0x2bc5	0x051f	unknown
0x1e10	0x4000	Point Grey Research
0x15aa	0x1555	Gearway Electronics

4.3 Method for Disabling CPU IDLE

Purpose: This can be used to analyze the prolonged CPU exit from IDLE, which causes slow USB interrupt processing, leading to issues such as UVC preview screen tearing and unstable UVC frame rates.

Taking the RK3588 EVB1 platform as an example:

```
cpunum=$(cat /proc/cpuinfo | grep processor | wc -l)
for i in `seq 0 $(($cpunum-1))`
do
echo 1 > /sys/devices/system/cpu/cpu$i/cpuidle/state0/disable
echo 1 > /sys/devices/system/cpu/cpu$i/cpuidle/state1/disable
done
```

Note: The "state*" nodes may vary across different platforms. For instance, RK3399 supports state0/1/2 nodes.

4.4 Method for Setting CPU to High Frequency

Purpose: This can be used to analyze and resolve issues with unstable UVC frame rates under high system load scenarios.

Modify the CPU scheduling policy to 'performance', which will keep the CPU running at the highest frequency to achieve maximum performance.

```
cpunum=$(cat /proc/cpuinfo | grep processor | wc -l)
for i in `seq 0 $(($cpunum-1))`
do
echo performance > /sys/devices/system/cpu/cpu$i/cpufreq/scaling_governor
done
```

4.5 Method for Setting DDR to High Frequency

Purpose: This can be used to analyze and resolve issues such as UVC preview failure, UVC preview screen tearing, or unstable UVC frame rates caused by DDR dynamic frequency variation.

Most Rockchip platforms support the DDR dynamic frequency scaling feature to optimize system power consumption. However, DDR dynamic frequency scaling may affect the following two UVC scenarios:

Scenario 1. At the moment of opening UVC preview, if DDR operates at a low frequency, it may probabilistically cause preview failure.

Scenario 2. During the UVC preview process, if DDR operates at a low frequency, it may probabilistically cause unstable UVC frame rates or screen tearing in compressed format images.

You can set DDR to operate at the highest frequency using the following two methods. The difference between Method 1 and Method 2 is: Method 1 is mainly used for the debugging phase, where DDR always operates at the highest frequency regardless of whether UVC is working or not. Method 2 is suitable for mass production, where DDR is set to always operate at the highest frequency only when UVC is working. When UVC is not working, DDR frequency scaling is still determined by system load.

Method 1. Set DDR to operate at the highest frequency via command line

Taking the RK3588 platform as an example:

```
console:/ # cd /sys/class/devfreq/dmc
console:/sys/class/devfreq/dmc # cat max_freq
2112000000
console:/sys/class/devfreq/dmc # echo userspace > governor
console:/sys/class/devfreq/dmc # echo 2112000000 > userspace/set_freq
```

Method 2. Set DDR to operate at high frequency by changing UVC driver settings [Recommended]

Linux-4.19 reference modification as follows:

```
From 970e4749125d685dc03ea32e3ae77e6f0ab7cff7 Mon Sep 17 00:00:00 2001
From: "william.wu" <william.wu@rock-chips.com>
Date: Sat, 11 Nov 2023 15:41:10 +0800
Subject: [PATCH] media: uvcvideo: set system status to performance when stream
 on

For rockchip platforms, set performance frequency for the
memory controller when uvc stream on, and clear it after
uvc stream off. It can improve uvc streaming stability.

Signed-off-by: william.wu <william.wu@rock-chips.com>
```

```
Change-Id: I1dc0cf10c552bba2c3a0f8a1bb37d90f546eb4a3
---
 drivers/media/usb/uvc/uvc_video.c | 6 ++++++
 1 file changed, 6 insertions(+)

diff --git a/drivers/media/usb/uvc/uvc_video.c b/drivers/media/usb/uvc/uvc_video.c
index ef3832b03709..d2158c94ae47 100644
--- a/drivers/media/usb/uvc/uvc_video.c
+++ b/drivers/media/usb/uvc/uvc_video.c
@@ -25,6 +25,8 @@
 #include <media/v4l2-common.h>

 #include "uvcvideo.h"
+#include <dt-bindings/soc/rockchip-system-status.h>
+#include <soc/rockchip/rockchip-system-status.h>

 /* ------------------------------------------------------------------------
  * UVC Controls
@@ -2149,6 +2151,7 @@ int uvc_video_enable(struct uvc_streaming *stream, int enable)
                }

                uvc_video_clock_cleanup(stream);
+               rockchip_clear_system_status(SYS_STATUS_PERFORMANCE);
                return 0;
        }

@@ -2161,6 +2164,8 @@ int uvc_video_enable(struct uvc_streaming *stream, int enable)
        if (ret < 0)
                goto error_commit;

+       rockchip_set_system_status(SYS_STATUS_PERFORMANCE);
+
        ret = uvc_init_video(stream, GFP_KERNEL);
        if (ret < 0)
                goto error_video;
@@ -2168,6 +2173,7 @@ int uvc_video_enable(struct uvc_streaming *stream, int enable)
        return 0;

 error_video:
+       rockchip_clear_system_status(SYS_STATUS_PERFORMANCE);
        usb_set_interface(stream->dev->udev, stream->intfnum, 0);
 error_commit:
        uvc_video_clock_cleanup(stream);
--
2.34.1
```

Linux-5.10 reference modification as follows:

```
From ab0546fdd01a8a8df0f15d0c648fa73aba76d173 Mon Sep 17 00:00:00 2001
From: William Wu <william.wu@rock-chips.com>
Date: Wed, 8 Nov 2023 09:49:48 +0800
Subject: [PATCH] media: uvcvideo: set system status to performance when stream
 on
```

```
For rockchip platforms, set performance frequency for the
memory controller when uvc stream on, and clear it after
uvc stream off. It can improve uvc streaming stability.

Signed-off-by: William Wu <william.wu@rock-chips.com>
Change-Id: I1dc0cf10c552bba2c3a0f8a1bb37d90f546eb4a3
---
 drivers/media/usb/uvc/uvc_video.c | 6 ++++++
 1 file changed, 6 insertions(+)

diff --git a/drivers/media/usb/uvc/uvc_video.c b/drivers/media/usb/uvc/uvc_video.c
index 03dfe96bceba..79ffe93b19ae 100644
--- a/drivers/media/usb/uvc/uvc_video.c
+++ b/drivers/media/usb/uvc/uvc_video.c
@@ -20,6 +20,8 @@
 #include <media/v4l2-common.h>

 #include "uvcvideo.h"
+#include <dt-bindings/soc/rockchip-system-status.h>
+#include <soc/rockchip/rockchip-system-status.h>

 /* ------------------------------------------------------------------------
  * UVC Controls
@@ -2139,6 +2141,8 @@ int uvc_video_start_streaming(struct uvc_streaming *stream)
        if (ret < 0)
                goto error_commit;

+       rockchip_set_system_status(SYS_STATUS_PERFORMANCE);
+
        ret = uvc_video_start_transfer(stream, GFP_KERNEL);
        if (ret < 0)
                goto error_video;
@@ -2146,6 +2150,7 @@ int uvc_video_start_streaming(struct uvc_streaming *stream)
        return 0;

 error_video:
+       rockchip_clear_system_status(SYS_STATUS_PERFORMANCE);
        usb_set_interface(stream->dev->udev, stream->intfnum, 0);
 error_commit:
        uvc_video_clock_cleanup(stream);
@@ -2176,4 +2181,5 @@ void uvc_video_stop_streaming(struct uvc_streaming *stream)
        }

        uvc_video_clock_cleanup(stream);
+       rockchip_clear_system_status(SYS_STATUS_PERFORMANCE);
 }
--
2.34.1
```

4.6 Method for Parsing UVC Image Frames

Function: Can be used to analyze image screen tearing issues in UVC preview or recording scenarios, assisting in locating whether the screen tearing phenomenon is an issue with the UVC peripheral or the UVC Host.

Taking the parsing of UVC MJPG format image frames (supporting high-bandwidth synchronous transmission) from Logitech UVC Camera (idVendor=046d, idProduct=0823) as an example, the method for capturing UVC data and parsing UVC image frames is as follows:

1. Use the Ellisys USB2.0 protocol analyzer to capture USB bus communication data during the UVC MJPG preview image process;
2. Refer to the steps in Figure 3, export the file in Text format (File -> Export -> Transactions data -> Text), and save the file;
3. Open the UVC txt file with a text editor, and refer to the UVC protocol specification[2] regarding the "Format of the Payload Header" to find the UVC frame header information. Refer to Figure 4, the UVC frame header length is 0x0C, and the UVC frame header information is 0x8C;
4. Use the Python script uvc-data.py to parse the UVC txt file, input the UVC frame header information, and after executing the script, it can automatically save UVC frame images in .jpg format;

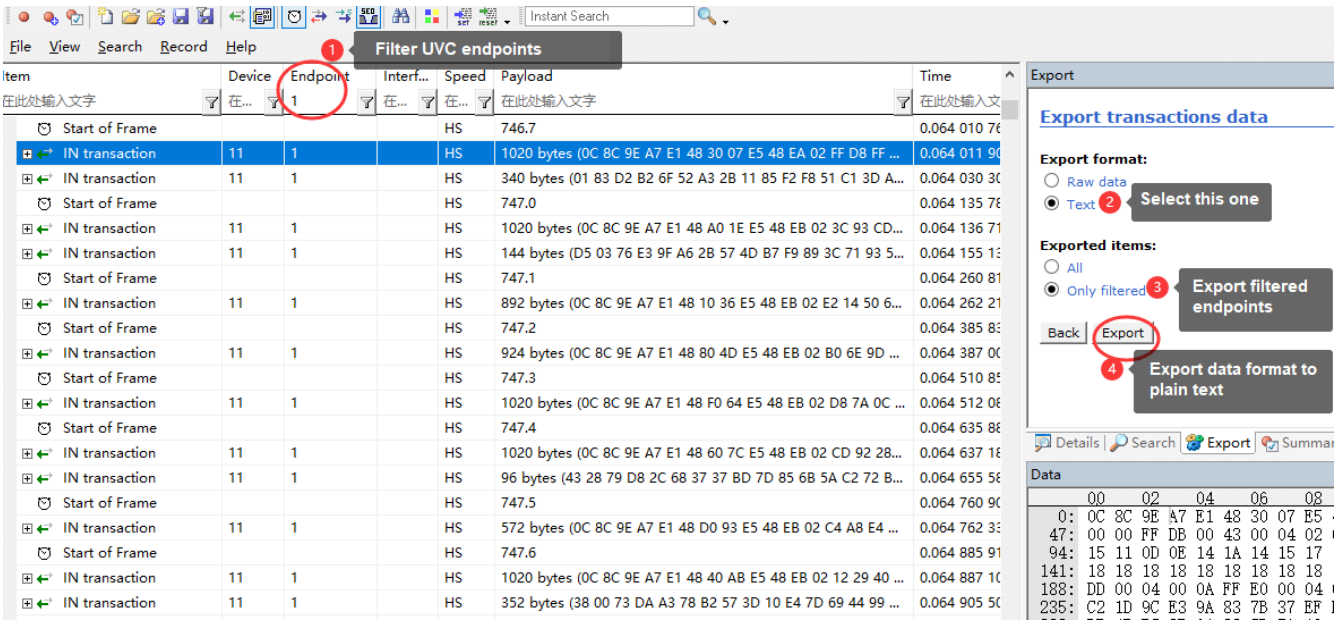


Figure 3 UVC Analyzer Data

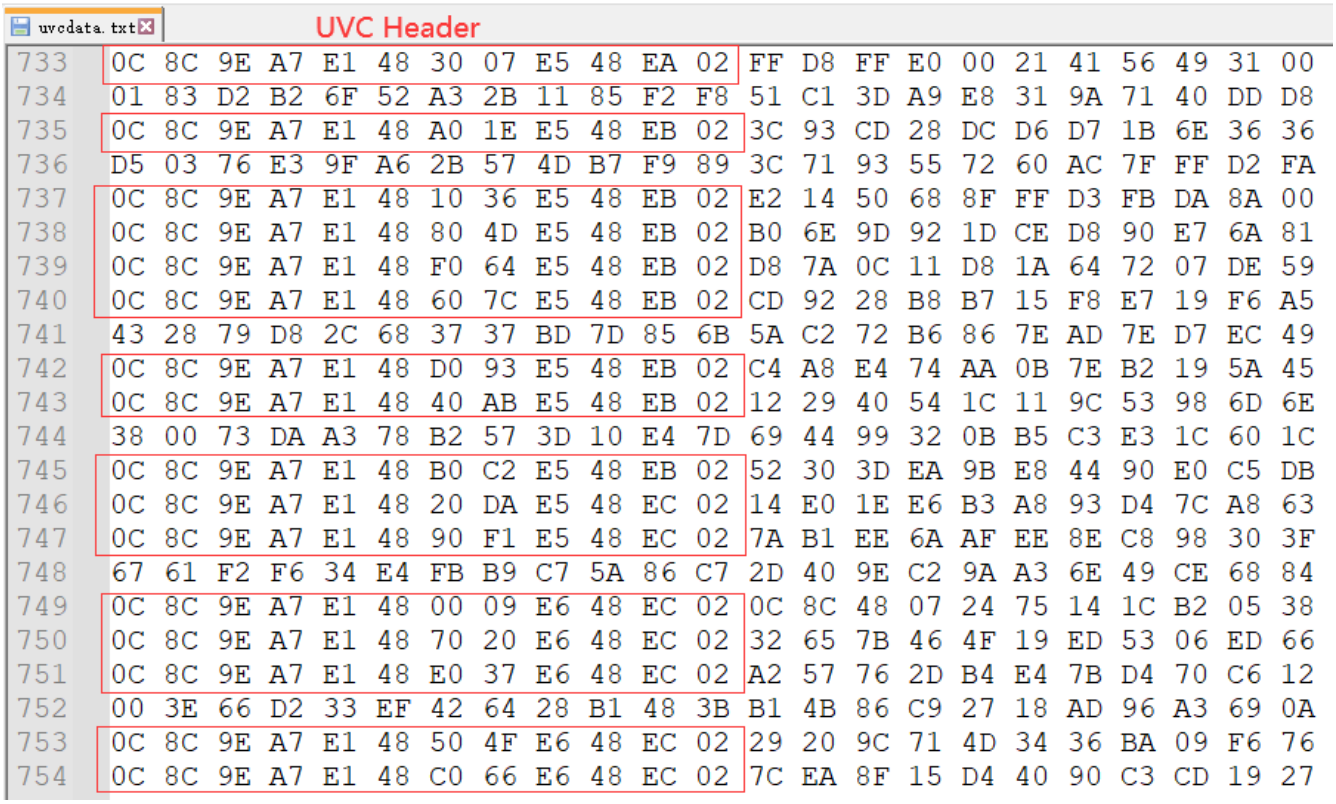


Figure 4 UVC Text Data

Python script for parsing UVC image frames uvc-data.py:

```
# -*- coding: utf-8 -*-
import argparse

parser = argparse.ArgumentParser()
parser.add_argument('--file', '-f', type=str, required=True, default='', help='The txt format
file of uvc data')
parser.add_argument('--header_len', '-l', type=str, required=True, default='', help='Length
of uvc header in bytes')
parser.add_argument('--header_info','-i', type=str, required=True, default='',
help='Information of uvc header')
parser.add_argument('--count', '-n', type=int, default=10, help='The count of images to
saved')
args = parser.parse_args()

def get_photo(cnt = 0):
    src = open(args.file, 'r')
    uvc_header_len = int(args.header_len, 16)
    uvc_header_info = int(args.header_info, 16)
    uvc_header_even_frame = uvc_header_info & 0xFC
    uvc_header_even_frame_eof = uvc_header_even_frame | 0x02
    uvc_header_odd_frame = (uvc_header_info & 0xFC) | 0x01
    uvc_header_odd_frame_eof = (uvc_header_odd_frame | 0x02)
    pic = []
    num = 0
    eof = 0
    uvc_first_frame = 0
    uvc_payload_offset = 0
```

```
    print('UVC Header Length:        ', hex(uvc_header_len), 'Bytes')
    print('UVC Header EVEN Frame:    ', hex(uvc_header_even_frame))
    print('UVC Header EVEN Frame EOF:', hex(uvc_header_even_frame_eof))
    print('UVC Header ODD Frame:     ', hex(uvc_header_odd_frame))
    print('UVC Header ODD Frame EOF: ', hex(uvc_header_odd_frame_eof))

    for line in src:
        line = line.split(' ')
        data0 = int(line[0], 16)
        data1 = int(line[1], 16)

        if ((data0 == uvc_header_len) and
            ((data1 == uvc_header_even_frame) or (data1 == uvc_header_odd_frame))):
            uvc_payload_offset = uvc_header_len
            uvc_first_frame = 1
        elif ((data0 == uvc_header_len) and
            ((data1 == uvc_header_even_frame_eof) or
            (data1 == uvc_header_odd_frame_eof) or
            (data1 == ((uvc_header_even_frame_eof & 0x0F) | 0x10)) or
            (data1 == ((uvc_header_odd_frame_eof & 0x0F) | 0x10)))):
            uvc_payload_offset = uvc_header_len
            uvc_first_frame = 0
            eof = 1
        else:
            uvc_payload_offset = 0
            uvc_first_frame = 0
            eof = 0

        if (eof == 1 and uvc_first_frame == 1):
            path = 'photo_' + str(num) + '.jpg'
            print(path + ' Saved ' + str(len(pic)) + 'Bytes')
            dst = open(path, 'wb+')
            dst.write(bytes.fromhex(''.join(pic)))
            dst.close()
            num += 1
            pic = []
            eof = 0
            if (cnt != 0 and cnt == num):
                break

        if (len(line) > uvc_payload_offset):
            for i in range(uvc_payload_offset, len(line)):
                pic.append(line[i])

    src.close()

get_photo(args.count)
```

Usage example:

```
# View the usage method of uvc-data.py
# -f: The uvc data text file to be parsed, required parameter;
```

```
# -l: The frame header length to be parsed, required parameter;
# -i: The uvc frame header information to be parsed, required parameter;
# -n: The number of uvc frame images to be parsed and saved;
python uvc-data.py -h
usage: uvc-data.py [-h] --file FILE --header_len HEADER_LEN --header_info HEADER_INFO [--
count COUNT]

options:
  -h, --help            show this help message and exit
  --file FILE, -f FILE  The txt format file of uvc data
  --header_len HEADER_LEN, -l HEADER_LEN
                        Length of uvc header in bytes
  --header_info HEADER_INFO, -i HEADER_INFO
                        Information of uvc header
  --count COUNT, -n COUNT
                        The count of images to saved
# Parse uvcdata.txt, uvc frame header length is 0x0c, uvc frame header information is 0x8c,
save 5 images
python uvc-data.py -f uvcdata.txt -l 0c -i 8c -n 5
UVC Header Length:         0xc Bytes
UVC Header EVEN Frame:     0x8c
UVC Header EVEN Frame EOF: 0x8e
UVC Header ODD Frame:      0x8d
UVC Header ODD Frame EOF:  0x8f
photo_0.jpg Saved 516573Bytes
photo_1.jpg Saved 827193Bytes
photo_2.jpg Saved 827585Bytes
photo_3.jpg Saved 3120193Bytes
photo_4.jpg Saved 188340Bytes
```

4.7 Methods for Optimizing UVC Driver

1. Increase the Number of UVC URBs

   Function: Enhance the UVC driver's tolerance to latency in response to USB controller interrupts.

   Impact: UVC URBs will buffer UVC frame data, increasing preview latency.

   Reference modification for Linux-4.19 is as follows:

```
diff --git a/drivers/media/usb/uvc/uvcvideo.h b/drivers/media/usb/uvc/uvcvideo.h
index 96632cc58d77..3761946af110 100644
--- a/drivers/media/usb/uvc/uvcvideo.h
+++ b/drivers/media/usb/uvc/uvcvideo.h
@@ -173,7 +173,7 @@
 #define DRIVER_VERSION         "1.1.1"

 /* Number of isochronous URBs. */
-#define UVC_URBS               5
+#define UVC_URBS               32
 /* Maximum number of packets per URB. */
 #define UVC_MAX_PACKETS                32
 /* Maximum status buffer size in bytes of interrupt URB. */
```

2. **Modify the UVC DMA Buffer Allocation Method**

   Function: Change the allocation method of UVC DMA buffer from the default usb_alloc_coherent to kmalloc, improving the efficiency of the UVC driver's software reading and parsing of buffer data.

   Impact: The USB driver must ensure the cache consistency of the DMA buffer. The Rockchip platform USB driver already supports this by default, requiring no additional modifications from the software.

```
diff --git a/drivers/media/usb/uvc/uvc_video.c b/drivers/media/usb/uvc/uvc_video.c
index d2158c94ae47..fff3c710bc97 100644
--- a/drivers/media/usb/uvc/uvc_video.c
+++ b/drivers/media/usb/uvc/uvc_video.c
@@ -1604,7 +1604,7 @@ static void uvc_free_urb_buffers(struct uvc_streaming *stream)
                struct uvc_urb *uvc_urb = &stream->uvc_urb[i];

                if (uvc_urb->buffer) {
-#ifndef CONFIG_DMA_NONCOHERENT
+#if 0
                        usb_free_coherent(stream->dev->udev, stream->urb_size,
                                          uvc_urb->buffer, uvc_urb->dma);
 #else
@@ -1651,7 +1651,7 @@ static int uvc_alloc_urb_buffers(struct uvc_streaming *stream,
                        struct uvc_urb *uvc_urb = &stream->uvc_urb[i];

                        stream->urb_size = psize * npackets;
-#ifndef CONFIG_DMA_NONCOHERENT
+#if 0
                        uvc_urb->buffer = usb_alloc_coherent(
                                stream->dev->udev, stream->urb_size,
                                gfp_flags | __GFP_NOWARN, &uvc_urb->dma);
@@ -1768,7 +1768,7 @@ static int uvc_init_video_isoc(struct uvc_streaming *stream,
                urb->context = uvc_urb;
                urb->pipe = usb_rcvisocpipe(stream->dev->udev,
                                            ep->desc.bEndpointAddress);
-#ifndef CONFIG_DMA_NONCOHERENT
+#if 0
                urb->transfer_flags = URB_ISO_ASAP | URB_NO_TRANSFER_DMA_MAP;
                urb->transfer_dma = uvc_urb->dma;
 #else
```

```
@@ -1834,7 +1834,7 @@ static int uvc_init_video_bulk(struct uvc_streaming *stream,

                usb_fill_bulk_urb(urb, stream->dev->udev, pipe, uvc_urb->buffer,
                                  size, uvc_video_complete, uvc_urb);
-#ifndef CONFIG_DMA_NONCOHERENT
+#if 0
                urb->transfer_flags = URB_NO_TRANSFER_DMA_MAP;
                urb->transfer_dma = uvc_urb->dma;
 #endif
```

5. UVC Common Troubleshooting Methods

5.1 UVC Preview Failure Issues

Issues with UVC preview not opening can generally be attributed to the following five scenarios:

- **Case Analysis 1:** USB High-Speed Handshake Failure

  The vast majority of UVC devices only operate in high-speed or super-speed modes. If signal issues cause the UVC to downgrade to full-speed mode, the UVC will not be able to preview normally.

  You can check the USB enumeration information through a serial port. If it prints similar information as "full-speed", it indicates that the high-speed handshake has failed.

  ```
  usb 3: new full-speed USB device number 3 using ohci-platform
  usb 3: not running at top speed; connect to a high speed hub
  ```

  Resolution: First, use an oscilloscope to analyze the high-speed handshake signal to identify the cause of the handshake failure. Then, solve the problem by optimizing hardware design and software Tuning PHY registers.

- **Case Analysis 2:** UVC Probe Control and Commit Control Bandwidth Negotiation Errors

  Before opening the preview, the Host and Device need to negotiate bandwidth. The normal negotiation process is shown in the figure below, which includes four stages:

  1. The Host first sends the desired settings to the USB device (PROBE);
  2. The device modifies the Host's desired settings within its own capabilities and returns them to the Host (PROBE);
  3. If the Host considers the settings feasible, it commits (COMMIT);
  4. The USB device sets the format, resolution, and bandwidth parameters based on the Host's Commit information;

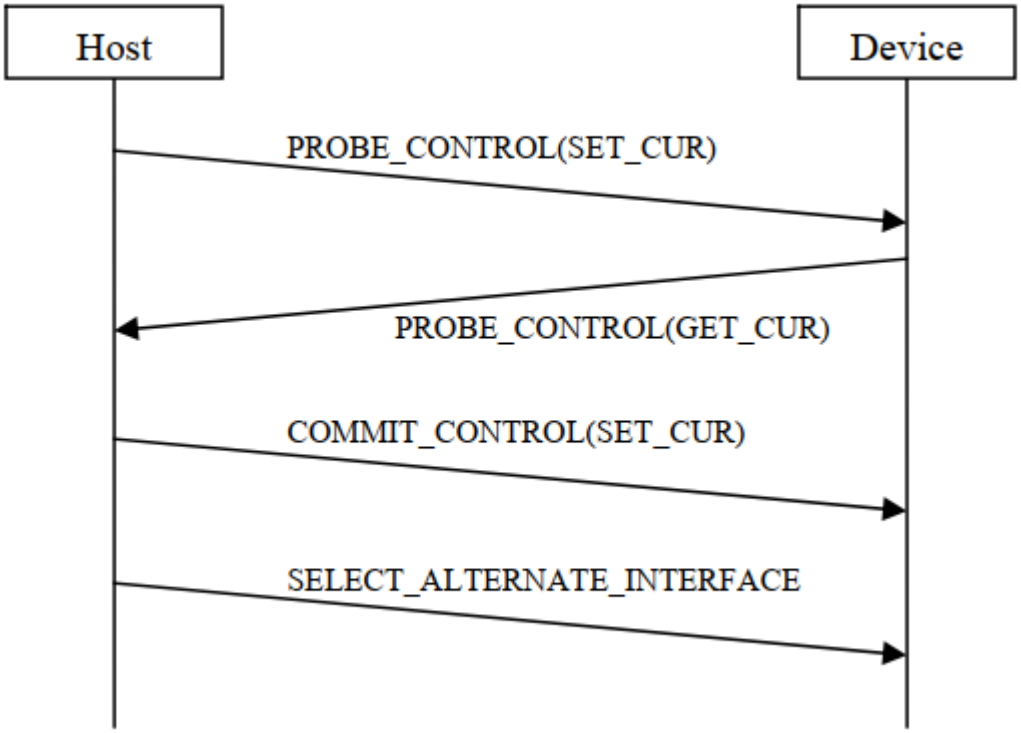


Figure 5 UVC Parameter Negotiation Process

Resolution: Use a USB protocol analyzer or usbmon tool to monitor the complete communication process of UVC enumeration and preview opening. Analyze the data from the analyzer to check if the UVC parameter negotiation process is correct. If it is confirmed to be a UVC Probe Control and Commit Control negotiation issue, modify the corresponding UVC driver logic or application logic.

- **Case Analysis 3:** UVC Host End Anomaly

  According to the USB 2.0 protocol specification, all USB transaction transfers are initiated by the USB Host, and the USB Device cannot actively send data to the Host. Therefore, if the Host end works abnormally, such as not executing the SetInterface command for streaming on when opening UVC preview, or not sending IN packet requests to the Device's Streaming endpoint after executing the SetInterface command, the Device will not be able to transmit the image stream.

  Resolution: You need to use a USB protocol analyzer to capture the complete transaction transfer on the USB bus to identify the problem. If a USB protocol analyzer is not available, it is recommended to refer to the section USB Monitoring and Analysis Tools to monitor the USB behavior of the Host end.

- **Case Analysis 4:** UVC autosuspend Compatibility Issues

  The UVC Host driver on the Rockchip platform defaults to enabling the auto suspend feature to optimize power consumption for UVC application scenarios. However, some UVC peripherals have compatibility issues with auto suspend on the Rockchip platform, leading to UVC preview anomalies.

  Resolution: Refer to the section Method for Disabling UVC Auto Suspend to analyze and resolve such issues.

- **Case Analysis 5:** DDR Frequency Variation Issues Causing UVC Stream Anomalies

Most Rockchip platforms support DDR dynamic frequency variation features to optimize system power consumption. When DDR operates at a lower frequency, it can cause higher bus latency for the USB controller. When the bus latency exceeds the tolerance value of UVC synchronous transmission, it may lead to anomalies in opening UVC previews.

Resolution: Refer to the section Method for Setting DDR to High Frequency to analyze and resolve these issues.

5.2 Unstable UVC Preview Frame Rate

There are multiple factors that can affect the frame rate of UVC previews, such as: USB controller interrupt response time, USB bus access latency, and the time taken by the UVC driver to parse UVC frames. In scenarios where the system is operating under high load, the instability of UVC frame rates becomes more pronounced.

Resolution: Refer to sections Method for Binding USB Interrupts to a Specific Core, Method for Disabling CPU IDLE, Method for Setting CPU to High Frequency, Method for Setting DDR to High Frequency to optimize USB interrupt response time and the latency of USB controller access to the bus, thereby enhancing CPU performance.

5.3 UVC Preview Artifacts

The primary cause of UVC preview artifacts is that most UVC peripherals transmit images using USB isochronous transfer, which is characterized by high real-time performance but is unreliable. If USB bus data transmission errors or losses occur, there is no retransmission. When analyzing UVC artifact issues, it is recommended to use a USB protocol analyzer to monitor the data transmission on the USB bus, which can accelerate the problem location.

The image formats transmitted by UVC include: uncompressed formats (YUV) and compressed formats (MJPG/H264/H265). These two image formats lead to different issues with UVC artifacts, and below are the analysis and troubleshooting methods for these two different image formats:

- **Case Analysis 1: UVC Transmitting Uncompressed Format Images with Preview Artifacts**

  In the UVC driver, the data size of a complete frame of uncompressed format images is verified. When the actual received image size differs from the theoretical value, the UVC driver will actively discard this image to avoid artifact phenomena caused by USB physical bus data transmission errors or UVC frame loss. Therefore, USB signal quality issues and USB controller driver issues in handling isochronous transfers can be ruled out first.

  Analysis Steps:

  1. Continuously capture the data communication of UVC preview uncompressed format images using a USB protocol analyzer, while observing the UVC preview artifact phenomenon. When the artifact phenomenon occurs, promptly stop the capture function of the USB protocol analyzer, save the protocol analyzer data, and then refer to the section Method for Parsing UVC Image Frames, using a Python script to automatically parse and save the image;
  2. If the data captured by the USB protocol analyzer is normal, and the image parsed from the analyzer data is also normal, it indicates that the UVC preview artifact issue is on the Host side.

Otherwise, it suggests that the source data transmitted by the UVC peripheral is already abnormal, and the Device side needs to be investigated;
3. If the issue is located on the Host side, it is recommended to focus on investigating the cache consistency of the USB controller DMA buffer;
4. You can also refer to the section v4l2-ctl Tool, using the v4l2-ctl tool to stream, and investigate whether it is related to the UVC application.

- **Case Analysis 2: UVC Transmitting Compressed Format Images with Preview Artifacts**

  In scenarios where UVC transmits compressed format images, artifact phenomena are more likely to occur because the image size of compressed formats is uncertain, and therefore, the UVC driver cannot verify the correctness of the actual received image size.

  Analysis Steps:

  1. Continuously capture the data communication of UVC preview compressed format images using a USB protocol analyzer, while observing the UVC preview artifact phenomenon. When the artifact phenomenon occurs, promptly stop the capture function of the USB protocol analyzer, save the protocol analyzer data, and then refer to the section Method for Parsing UVC Image Frames, using a Python script to automatically parse and save the image;
  2. If the USB protocol analyzer observes obvious transmission errors, such as: CRC error, incomplete data packets, etc., then focus on investigating USB signal quality issues;
  3. If the image parsed by the USB protocol analyzer also shows abnormal image display, it indicates that the data transmitted on the USB physical bus is already abnormal. Further find the USB analyzer data corresponding to the abnormal image, if it is found that the Host has not issued an IN packet request within multiple consecutive microframe intervals, it suggests a relationship with the Host side USB controller not handling isochronous transfers in time. If the Host transmission behavior is normal, it indicates that the source data transmitted by the Device side may have issues, and further investigation of the Device side is needed;
  4. If the image parsed by the USB protocol analyzer is normal, further investigate the cache consistency of the Host side USB controller DMA buffer, USB/CPU bus priority (QOS), UVC application processing, and other influencing factors;
  5. You can refer to the section v4l2-ctl Tool, using the v4l2-ctl tool to stream, and investigate whether it is related to the UVC application.

  Resolution Methods:

  1. USB signal quality issues: Test the USB HS eye diagram, and based on the eye diagram test report, optimize the hardware design and modify the PHY parameter configuration in the software driver[3];
  2. USB controller not handling isochronous transfers in time: Refer to the sections Method for Binding USB Interrupts to a Specific Core, Method for Disabling CPU IDLE, Method for Setting CPU to High Frequency, Method for Setting DDR to High Frequency, optimize the USB interrupt response time and the latency of the USB controller accessing the bus, and improve CPU performance;
  3. Increase the bus priority (QOS) of USB/CPU.

5.4 Insufficient Bandwidth Issues in Multi-UVC Scenarios

Insufficient UVC bandwidth issues typically occur in the following application scenarios:

A USB Host interface connects two or more USB2.0 UVC Cameras through a HUB, and fails to open the preview, with the kernel indicating an error log:

1. USB3.0 Host error log

```
usb 5-1.4: Not enough bandwidth for new device state.
usb 5-1.4: Not enough bandwidth for altsetting 11
VIDIOC_STREAMON: failed: No space left on device
```

2. USB2.0 Host error log

```
uvcvideo: Failed to submit URB 0 (-28).
VIDIOC_STREAMON: failed: No space left on device
```

- **Case Analysis:** RK3588 USB2.0 Host expands USB ports through a HUB, with two Logitech USB Cameras connected to the downstream port of the HUB: Logitech HD Network Camera C930 and UVC Camera (046d:0823).

  Method for reproducing the issue:

```
# step1. Check UVC video node information
console:/ # v4l2-ctl --list-devices
Logitech HD Network Camera C930 (usb-fc880000.usb-1.1):
        /dev/video21
        /dev/video22

UVC Camera (046d:0823) (usb-fc880000.usb-1.4):
        /dev/video23
        /dev/video24

# step2. Two UVC streams simultaneously at a resolution of 1920*1080 in MJPG format
console:/ # v4l2-ctl --device /dev/video21 --set-fmt-
video=width=1920,height=1080,pixelformat=MJPG --stream-mmap &
console:/ # v4l2-ctl --device /dev/video23 --set-fmt-
video=width=1920,height=1080,pixelformat=MJPG --stream-mmap &

# step3. video21 stream successful, but video23 stream failed, the serial port prompted a
log
uvcvideo: Failed to submit URB 0 (-28).
VIDIOC_STREAMON: failed: No space left on device
```

  Analysis of the issue:

```
# Check UVC device endpoint information
console:/ # cat /sys/kernel/debug/usb/devices
# Corresponding to Logitech HD Network Camera C930 device
T:  Bus=02 Lev=02 Prnt=02 Port=00 Cnt=01 Dev#=  3 Spd=480  MxCh= 0
D:  Ver= 2.00 Cls=ef(misc ) Sub=02 Prot=01 MxPS=64 #Cfgs=  1
P:  Vendor=046d ProdID=0891 Rev= 0.19
S:  Product=Logitech HD Network Camera C930
S:  SerialNumber=83FBB5EE
......
```

```
I:* If#= 1 Alt= 0 #EPs= 0 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
I:  If#= 1 Alt= 1 #EPs= 1 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
E:  Ad=81(I) Atr=05(Isoc) MxPS= 192 Ivl=125us
I:  If#= 1 Alt= 2 #EPs= 1 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
......
I:  If#= 1 Alt=10 #EPs= 1 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
E:  Ad=81(I) Atr=05(Isoc) MxPS=2688 Ivl=125us
I:  If#= 1 Alt= 11 #EPs= 1 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
E:  Ad=81(I) Atr=05(Isoc) MxPS=3072 Ivl=125us

# Corresponding to UVC Camera (046d:0823) device
T:  Bus=02 Lev=02 Prnt=02 Port=03 Cnt=02 Dev#=  4 Spd=480  MxCh= 0
D:  Ver= 2.00 Cls=ef(misc ) Sub=02 Prot=01 MxPS=64 #Cfgs=  1
P:  Vendor=046d ProdID=0823 Rev= 0.10
S:  SerialNumber=7F65EA20
......
I:* If#= 3 Alt= 0 #EPs= 0 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
I:  If#= 3 Alt= 1 #EPs= 1 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
E:  Ad=81(I) Atr=05(Isoc) MxPS= 192 Ivl=125us
......
E:  Ad=81(I) Atr=05(Isoc) MxPS=1984 Ivl=125us
I:  If#= 3 Alt= 10 #EPs= 1 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
E:  Ad=81(I) Atr=05(Isoc) MxPS=3060 Ivl=125us
I:  If#= 3 Alt= 11 #EPs= 1 Cls=0e(video) Sub=02 Prot=00 Driver=uvcvideo
E:  Ad=81(I) Atr=05(Isoc) MxPS=3060 Ivl=125us
```

From the above endpoint information, it can be seen that both Logitech USB Cameras support dynamic bandwidth configuration, with the smallest Max packet size being 192 bytes, and the largest Max packet size being 3072 bytes for one and 3060 bytes for the other. The Max packet size here refers to the maximum packet size that the USB Camera can transmit within a USB microframe. Dynamic bandwidth configuration means that the USB Camera will dynamically adjust the Max packet size based on the image resolution and format.

Refer to the section UVC Driver Debug Interface to dynamically enable trace information of the UVC driver to determine the bandwidth configuration used when UVC opens the preview.

```
# Check the USB bus bandwidth information occupied by the UVC device
console:/ # echo 0xffff > /sys/module/uvcvideo/parameters/trace
console:/ # echo 8 > /proc/sys/kernel/printk
console:/ # v4l2-ctl --device /dev/video21 --set-fmt-
video=width=1920,height=1080,pixelformat=MJPG --stream-mmap

# Key trace information for Logitech HD Network Camera C930 device is as follows:
uvcvideo: Selecting alternate setting 11 (3072 B/frame bandwidth).

# Similarly, for UVC Camera (046d:0823) device, the key trace information is as follows:
uvcvideo: Selecting alternate setting 11 (3060 B/frame bandwidth).
```

From the above uvc trace information, it can be seen that when streaming at a resolution of 1920*1080 in MJPG format, the Logitech HD Network Camera C930 device requires 3072 B/frame of bandwidth, while the UVC Camera (046d:0823) device also requires 3060 B/frame of bandwidth. Here, the frame corresponds to the microframe defined by the USB protocol (frame interval 125 microseconds).

According to the USB2.0 protocol specification[4]:

Section 5.6.4 Isochronous Transfer Bus Access Constraints states:

High-speed endpoints can allocate at most **80%** of a microframe for periodic transfers.

This means the maximum microframe time occupied by periodic transfers = 125 us * 80% = **100us**, with the remaining time reserved for non-periodic transfer devices.

Section 5.11.3 Calculating Bus Transaction Times states:

```
High-speed (Input)
Isochronous Transfer (No Handshake)
= (38 * 8 * 2.083) + (2.083 * Floor(3.167 + BitStuffTime(Data_bc))) + Host_Delay
```

The Linux kernel USB driver is strictly implemented in compliance with the USB 2.0 specification, and the corresponding code implementation is shown as follows:

```
include/linux/usb/hcd.h
/*
 * Ceiling [nano/micro]seconds (typical) for that many bytes at high speed
 * ISO is a bit less, no ACK ... from USB 2.0 spec, 5.11.3 (and needed
 * to preallocate bandwidth)
 */
#define USB2_HOST_DELAY 5       /* nsec, guess */
#define HS_NSECS(bytes) (((55 * 8 * 2083) \
        + (2083UL * (3 + BitTime(bytes))))/1000 \
        + USB2_HOST_DELAY)
#define HS_NSECS_ISO(bytes) (((38 * 8 * 2083) \
        + (2083UL * (3 + BitTime(bytes))))/1000 \
        + USB2_HOST_DELAY)
#define HS_USECS(bytes)         NS_TO_US(HS_NSECS(bytes))
#define HS_USECS_ISO(bytes)     NS_TO_US(HS_NSECS_ISO(bytes))

drivers/usb/core/hcd.c
long usb_calc_bus_time (int speed, int is_input, int isoc, int bytecount)
{
        unsigned long   tmp;
        ......
        case USB_SPEED_HIGH:    /* ISOC or INTR */
                /* FIXME adjust for input vs output */
                if (isoc)
                        tmp = HS_NSECS_ISO (bytecount);
                else
                        tmp = HS_NSECS (bytecount);
        ......
}
```

Using the above calculation method, the bus times corresponding to different ISOC maximum packet sizes are listed below:

Max packet size (bytes/uframe)	bus time(us)
256	5
640	13
1024	20
1920	37
3072	60

Therefore, when two UVC devices simultaneously transmit video streams at high bandwidth (e.g., 3072 bytes per microframe), the consumed bus time will exceed the 100 µs upper limit defined by the USB 2.0 specification, resulting in insufficient bandwidth for UVC.

Solutions: Method 1. Optimize the bandwidth allocation policy of the UVC driver (Recommended Priority)
Function: Reduce the maximum bandwidth request of the target UVC device Impact: May lower the frame rate of uncompressed formats (YUV)

Reference changes for Linux 5.10 are as follows:

```
diff --git a/drivers/media/usb/uvc/uvc_video.c b/drivers/media/usb/uvc/uvc_video.c
index f6373d678d25..ee798b778255 100644
--- a/drivers/media/usb/uvc/uvc_video.c
+++ b/drivers/media/usb/uvc/uvc_video.c
@@ -1845,6 +1845,7 @@ static int uvc_video_start_transfer(struct uvc_streaming *stream,
        struct usb_interface *intf = stream->intf;
        struct usb_host_endpoint *ep;
        struct uvc_urb *uvc_urb;
+       struct usb_device *udev = interface_to_usbdev(intf);
        unsigned int i;
        int ret;

@@ -1866,6 +1867,13 @@ static int uvc_video_start_transfer(struct uvc_streaming *stream,
                /* Isochronous endpoint, select the alternate setting. */
                bandwidth = stream->ctrl.dwMaxPayloadTransferSize;

+               if ((le16_to_cpu(udev->descriptor.idVendor) == 0x2bdf) &&
+                   (le16_to_cpu(udev->descriptor.idProduct) == 0x0293) &&
+                   (bandwidth > 1600)){
+                       printk("UVC_DBG: limit bandwidth from %u to 1600B\n", bandwidth);
+                       bandwidth = 1600;
+               }
+
                if (bandwidth == 0) {
                        uvc_trace(UVC_TRACE_VIDEO, "Device requested null "
                                "bandwidth, defaulting to lowest.\n");
```

Method 2. Increase the bandwidth proportion allocated for periodic transfers on the USB bus

Function: Raise the percentage of USB bus bandwidth reserved for periodic transfers up to 95% or higher to boost available bandwidth for periodic traffic.

Impact: Other non-periodic transfer devices on the same USB bus (e.g., USB flash drives) may malfunction.

Limitation: This method is not applicable to USB 3.0 xHCI host controllers.

Reference driver modifications for USB 2.0 EHCI host controllers are listed below:

```
diff --git a/drivers/usb/host/ehci-hcd.c b/drivers/usb/host/ehci-hcd.c
index 8aff19ff8e8f..bed879a4ab5b 100644
--- a/drivers/usb/host/ehci-hcd.c
+++ b/drivers/usb/host/ehci-hcd.c
@@ -475,7 +475,7 @@ static int ehci_init(struct usb_hcd *hcd)
         * by default set standard 80% (== 100 usec/uframe) max periodic
         * bandwidth as required by USB 2.0
         */
-       ehci->uframe_periodic_max = 100;
+       ehci->uframe_periodic_max = 125;

        /*
         * hw default: 1K periodic list heads, one per frame.
```

Reference changes for the DWC2 USB 2.0 Host controller are as follows:

```
diff --git a/drivers/usb/dwc2/hcd_queue.c b/drivers/usb/dwc2/hcd_queue.c
index b2e0721a3eb8..e8b394bd9aae 100644
--- a/drivers/usb/dwc2/hcd_queue.c
+++ b/drivers/usb/dwc2/hcd_queue.c
@@ -119,7 +119,7 @@ static int dwc2_check_periodic_bandwidth(struct dwc2_hsotg *hsotg,
                 * High speed mode
                 * Max periodic usecs is 80% x 125 usec = 100 usec
                 */
-               max_claimed_usecs = 100 - qh->host_us;
+               max_claimed_usecs = 125 - qh->host_us;
        } else {
                /*
                 * Full speed mode
```

5.5 UVC Preview Latency Issues

The transmission chain of UVC images is quite lengthy, and the latency at each module within the transmission chain can potentially affect the total latency of the preview. Therefore, when analyzing UVC preview latency issues, it is necessary to first use a USB protocol analyzer to capture the complete data stream during the preview startup phase to identify whether the latency bottleneck is on the Device side or the Host side. If it is a Host side issue, then further decompose the latency of each module in the Host side transmission chain for UVC image streams.

- **Case Analysis:** Host side caching images leading to structured light module MJPEG 1080P latency issues

  The reason is that after the Host side APK libuvc receives image data from the kernel, it performs a memcopy operation, placing it into the frame queue managed by the application itself. This frame queue is asynchronous with the kernel's urb (5 urbs are used for rotating reception of image data). Since the Host side's 1080P software decoding and display are time-consuming, the first frame of the

image is only displayed after the kernel has received 6-7 frames of images and the APK libuvc has received all of them and placed them into the frame queue. The Host side's frame queue caches several frames of images, leading to latency issues.

Solution: Improve the efficiency of Host side image software decoding to avoid image caching.

5.6 UVC EMI Issues

- **Case Analysis:** EMI issues are typically related to the hardware circuit design of USB devices. When EMI problems occur, they can affect the normal communication of UVC, such as: UVC cannot open the preview or the preview image is frozen.

  If the USB Host side is using the Linux kernel, when an EMI issue occurs, the Host kernel generally prints the following log:

  ```
  usb usb1-port1: disabled by hub (EMI?), re-enabling...
  ```

  It should be noted that the principle of Linux USB Host's software driver detecting EMI is: the USB Host controller (such as the EHCI controller) will continuously poll the port enable status of the register. When the software detects that the port status of the USB controller suddenly changes from enable to disable, and the register's peripheral connection status still shows as connected, the software side will consider it as an EMI error causing the controller status to be abnormal and print the above EMI log. Therefore, the software side's indication of EMI errors can only serve as an important direction for troubleshooting problems, but it cannot definitively confirm that it is caused by EMI issues.

  Resolution Methods:

  1. Use an oscilloscope to test the USB eye diagram and generate a test report for hardware engineers and USB driver software engineers. Engineers can modify the USB hardware circuit and adjust the USB PHY registers to improve USB signal quality, thereby improving EMI issues.
  2. Use an oscilloscope to test the DP/DM single-ended and differential signals. If the noise is too large, it will affect the quality of the USB DP/DM signals and trigger EMI issues. The noise problem can be improved through the following three hardware optimization methods:

     (1) Series common-mode inductor for DP/DM;

     (2) Increase the common ground connection between Device and Host;

     (3) Replace with USB Host sockets that have a larger contact area for DP/DM Pins.

6. References

1. kernel/Documentation/trace/ftrace.rst
2. USB Video Class v1.1
3. Rockchip Introduction USB SQ Tool
4. USB 2.0 Specification